

Make vs CMake

El duelo que todo dev de C/C++ necesita ver antes de arrancar un proyecto



Make

El veterano de los 70s

+ LO QUE TIENE DE BUENO

- ✓ Viene instalado en casi cualquier Unix/Linux sin hacer nada
- ✓ Sintaxis directa: regla → dependencias → comandos
- ✓ Perfecto para proyectos chicos o scripts simples
- ✓ Documentación y ejemplos por todos lados
- ✓ Sirve para cualquier tarea, no solo compilar

+ SUS PUNTOS DÉBILES

- ✗ Los Makefiles crecen y se vuelven un monstruo ilegible
- ✗ Portabilidad entre sistemas operativos: un dolor de cabeza
- ✗ Manejo de dependencias externas básicamente manual
- ✗ Los tabs obligatorios en la sintaxis te van a odiar
- ✗ Sin soporte nativo para IDEs modernos

+ CARACTERÍSTICAS CLAVE

Build tool directo

Reglas y targets

Shell commands

Desde 1976

Nivel bajo



CMake

El meta-build system moderno

+ LO QUE TIENE DE BUENO

- ✓ Genera Makefiles, Ninja, proyectos de VS, Xcode y más
- ✓ Multiplataforma desde el diseño (Windows, Mac, Linux)
- ✓ Manejo de dependencias con FetchContent o find_package
- ✓ Integración nativa con CLion, VS Code, Visual Studio
- ✓ Escala muy bien en proyectos grandes con submódulos

+ SUS PUNTOS DÉBILES

- ✗ La sintaxis de CMakeLists.txt tiene curva de aprendizaje
- ✗ Para proyectos sencillos es un cañón contra moscas
- ✗ Mensajes de error crípticos que hacen sufrir
- ✗ CMake antiguo vs moderno (target-based) genera confusión
- ✗ Hay que instalarlo aparte — no viene de fábrica

+ CARACTERÍSTICAS CLAVE

Meta-build system

Multiplataforma

CMakeLists.txt

Desde 2000

Nivel alto

¿CUÁNDO USAR CADA UNO?

Usa Make cuando...

- 👤 Es un proyecto solo para Linux/Unix
- 🔧 Es pequeño y sin muchas dependencias
- ⚡ Necesitas algo rápido sin instalar nada
- 🔧 Automatizas tareas: tests, clean, deploy
- 👤 Estás aprendiendo cómo funciona el build

Usa CMake cuando...

- 🌐 El código debe correr en varios SO
- 👤 El proyecto es mediano o grande
- 🔧 Necesitas integración con IDE
- 🔧 Tienes librerías externas que manejar
- 👤 Trabaja un equipo con distintos entornos